1 牛顿法

1.1 泰勒级数

- 泰勒级数展开

$$
\begin{aligned}
f(x) &= \lim_{n\to\infty}\sum_{i=1}^{n}\frac{1}{n!}f^{(n)}(x_0)(x-x_0)^n \\
&= f(x_0)+f'(x_0)(x-x_0)+\frac{f''(x_0)}{2!}(x-x_0)^2+\cdots+\frac{1}{n!}f^n(x_0)(x-x_0)^n \\
&\quad + O\left[(x-x_0)^n\right]/\frac{f^{(n+1)(\xi)}}{(n+1)!}(x-x_0)^{n+1}
\end{aligned} \tag{1}
$$

- 麦克劳林级数展开

  泰勒展开式在 0 处展开

$$
\begin{aligned}
f(x) &= \lim_{n\to\infty}\sum_{i=1}^{n}\frac{1}{n!}f^{(n)}(0)x^n \\
&= f(0)+f'(0)x+\frac{f''(0)}{2!}x^2+\cdots+\frac{1}{n!}f^n(0)x^n \\
&\quad + O\left(x^n\right)/\frac{f^{(n+1)(\xi)}}{(n+1)!}x^{n+1}
\end{aligned} \tag{2}
$$

  其中

  1. $f^{(n)}$：表示对函数f求 n 阶导数；
  2. $O\left(x^n\right)$：为佩亚诺余项，代表x^n的高阶无穷小，要求 $f(x)$n 阶可导；
  3. $\frac{f^{(n+1)(\xi)}}{(n+1)!}x^{n+1}$：为拉格朗日型余项，要求 $f(x)$ n+1 阶可导。

1.2 牛顿法

1.2.1 原理（一维情况）

- 假如已知函数 $f(x)$, 想要求 $f(x)=0$ 的解 (或者叫根)。
  `牛顿法（Newton's method）大致的思想` 是:
  1. 选一个初始位置 x_0 （这个位置最好是在根的附近）；
  2. 在这个位置上找一个 $f(x)$ 的近似函数（通常用泰勒展开 Q ）；
  3. 令近似函数为 0 , 求解;
  4. 以这个解为新的位置 x_1;
  5. 重复上述迭代, 到第 n 次迭代得到 x_n ， 当 $|x_n-x_{n-1}|$ 足够小, 结束。 x_n 就是 $f(x)=0$ 的近似解。

- 牛顿法思想：使用 $f(x)$ 的泰勒展开式（前几项）

$$f(x) \approx f(x_0) + f'(x_0) \tag{3}$$

不断迭代来近似寻找方程$f(x) = 0$ 的根。

设第一次迭代在 x_0 处，则有

$$\begin{aligned} f(x) &= 0 \\ \Rightarrow f(x_0) + f'(x_0)(x - x_0) &= 0 \\ f'(x_0)(x_0 - x) &= f(x_0) \\ x_0 - x &= \frac{f(x_0)}{f'(x_0)} \\ x &= x_0 - \frac{f(x_0)}{f'(x_0)} \end{aligned} \tag{4}$$

则$f(x) = 0$ 第一次迭代的近似解x_1为

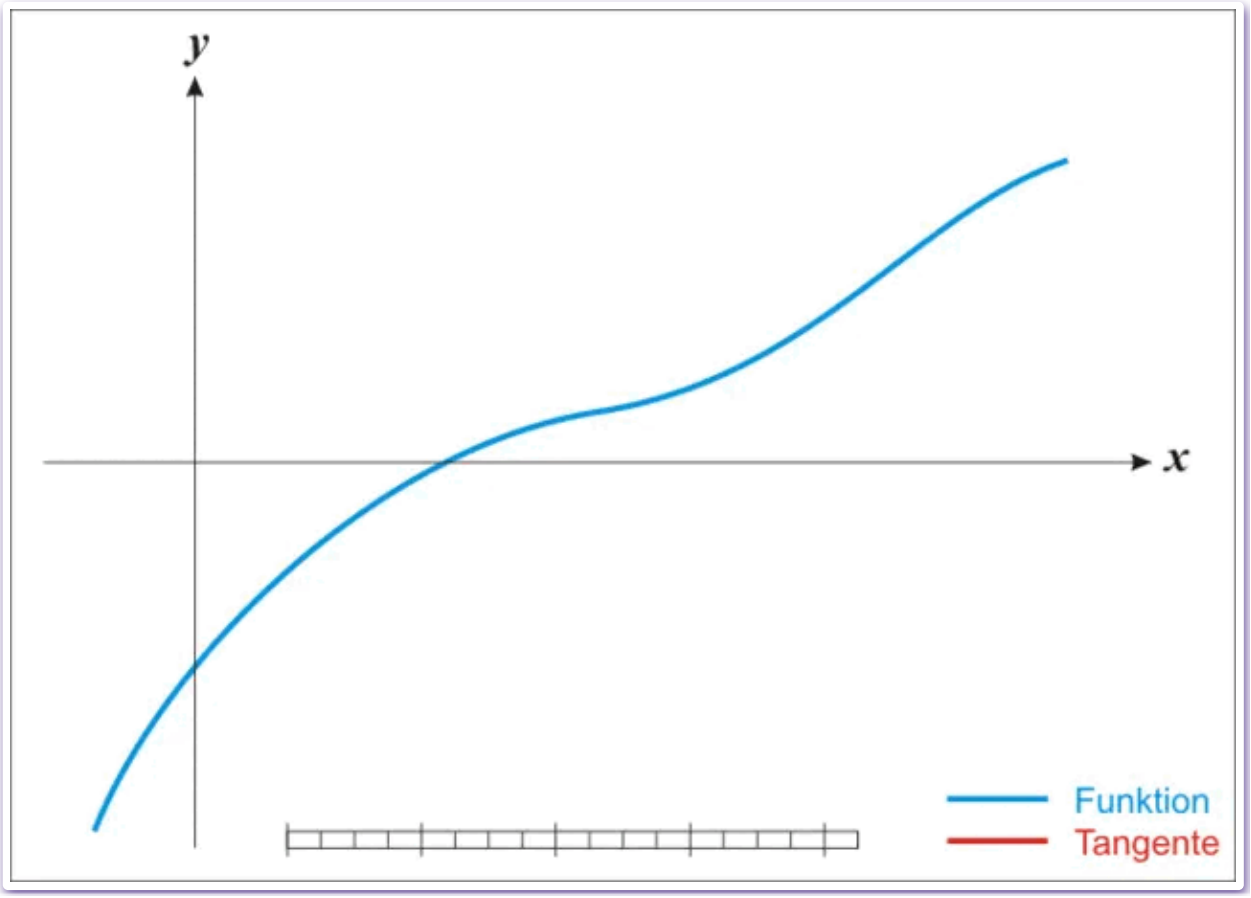
$$x_1 = x_0 - \frac{f(x_0)}{f'(x_0)} \tag{5}$$

由此得到第 n+1 次的迭代解为

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)} \tag{6}$$

由于对 $f(x)$ 的近似只是一阶展开, 因此 x_1 并非 $f(x) = 0$ 的解, 只能说 $f(x_1)$ 比 $f(x_0)$ 更接近0。

- 迭代过程图（维基百科）



- 牛顿法一维情况

  迭代公式

$$x_{n+1} = x_n - \frac{f'(x_0)}{f''(x_0)} \tag{7}$$

  牛顿法的推导基于二阶可微函数的泰勒展开

$$\begin{gathered} f(x) = 0 \\ \Rightarrow f(x_0) + f'(x_0)(x - x_0) + \frac{f''(x_0)}{2!}(x - x_0)^2 = 0 \\ \text{两边求导} \\ f'(x_0) + f''(x_0)(x - x_0) = 0 \\ f''(x_0)(x_0 - x) = f'(x_0) \\ x_0 - x = \frac{f'(x_0)}{f''(x_0)} \\ x = x_0 - \frac{f'(x_0)}{f''(x_0)} \end{gathered} \tag{8}$$

1.2.2 求解最优化问题（高维情况）

- 对于无约束最优化问题 $\min_{x \in \mathbf{R}^n} f(x)$ ，可根据极小点的必要条件 $\nabla f(x) = 0$ 采用牛顿法求解：

$$x_{k+1} = x_k - H_k^{-1} g_k \tag{9}$$

  其中

  1. $g_k = g(x_k) = \nabla f(x_k)$ 是 $f(x)$ 的梯度向量在点 x_k 的值;
  2. $H_k = H(x_k), H(x) = \left[\frac{\partial^2 f}{\partial x_i \partial x_j}\right]_{n \times n}$ 是 $f(x)$ 的海塞矩阵(二阶偏导数矩阵)。

$$H(f) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \cdots & \frac{\partial^2 f}{\partial x_2 \partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \frac{\partial^2 f}{\partial x_n \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix} \tag{10}$$

- 具体步骤

  输入： 目标函数 $f(x)$, 梯度 $g(x) = \nabla f(x)$, 海塞矩阵 $H(x)$, 精度要求 ϵ ；
  输出 : $f(x)$ 的极小点 x^* 。

  1. 取初始点 x_0, 置 $k = 0$
  2. 计算 g_k, 若 $\|g_k\| < \epsilon$, 则 $x^* = x_k$, 停止计算; 否则转 (3)
  3. 计算 H_k, 令 $x_{k+1} = x_k - H_k^{-1} g_k$
  4. 置 $k = k + 1$ ， 转 (2)

备注: 第 (3) 步中, 涉及到 H_k^{-1} 的计算, 实际应用中, 通常并不直接对 H_k 进行求逆, 而是将其转化为求解线性代数方程组 $H_k d_k = -g_k$, 此时可根据系数矩阵 H_k 的性态来选择合适的迭代法, 如预条件共轭梯度法（PCG）、代数多重网格法 (AMG) 等。

1.2.3 小结

- 当目标函数是二次函数时, 海塞矩阵退化成一个常数矩阵, 从任一初始点出发, 牛顿法可一步到达, 因此它是一种具有二次收玫性的算法。对于非二次函数, 若函数的二次性态较强, 或迭代点已进入极小点的邻域, 则其收敛速度也是很快的, 这是牛顿法的主要优点。

  牛顿法的迭代公式中由于没有步长因子, 是定步长迭代, 对于非二次型目标函数, 有时会使函数值上升, 即出现 $f(x_{k+1}) > f(x_k)$ 的情况, 更甚者, 可能出现迭代点列 $\{x_k\}$ 发散而导致计算失败的情况。为解决这个问题, 出现了“阻尼牛顿法”, 增加一个步长因子 λ_k, 将算法流程 (3) 中的计算公式修改为:

$$x_{k+1} = x_k - \lambda_k H_k^{-1} g_k \tag{11}$$

  牛顿法的另一个弊病在于, 每一次迭代都要计算 H^{-1}, 这一步计算比较复杂, 拟牛顿法将解决这个问题。

1.3 拟牛顿法

- 暂时略过

2 优化问题相关

- 李沐-优化算法

> https://www.bilibili.com/video/BV1bP4y1p7Gq?vd_source=7937b7ae341caaf55cd0ac02b03193a1

- b站

> https://www.bilibili.com/video/BV1t47TzfE8R?vd_source=7937b7ae341caaf55cd0ac02b03193a1

2.1 优化问题

- 一般形式

$$\textbf{minimize} \quad f(x) \quad subjectto \mathbf{x} \in C \tag{12}$$

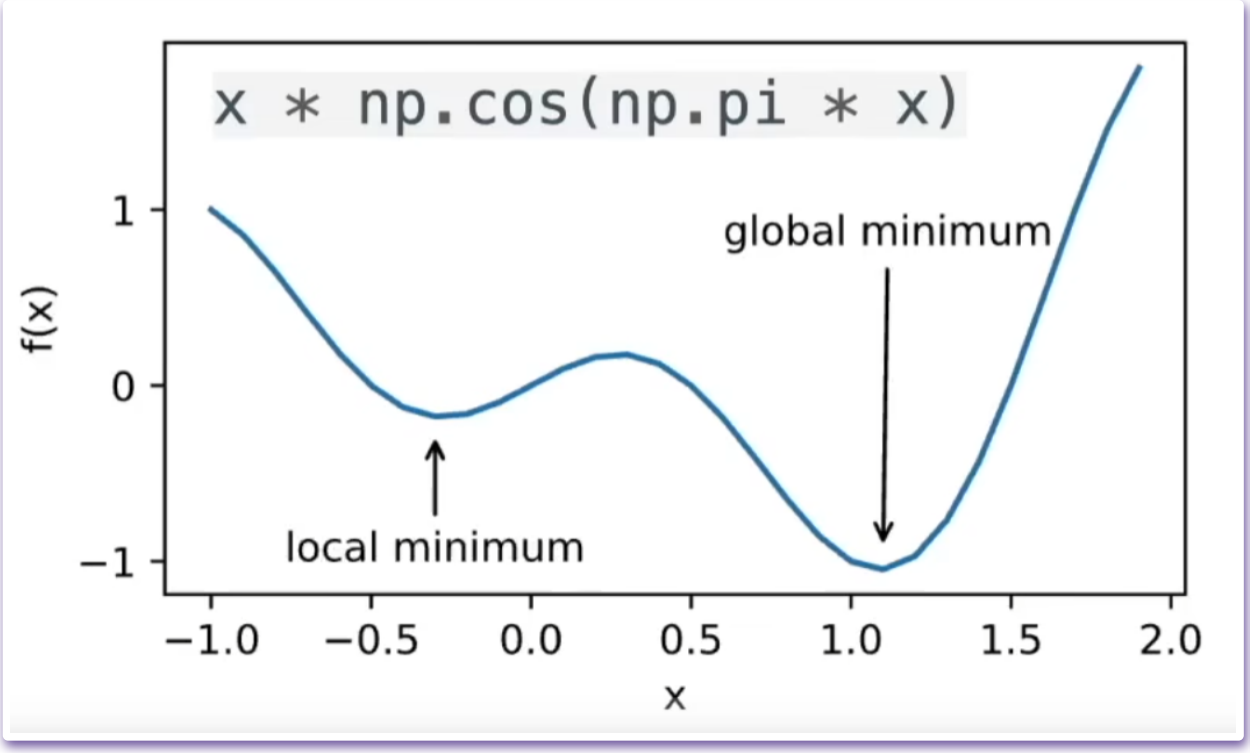
- 目标函数 $f : \mathbb{R}^n \to \mathbb{R}$
- 限制集合例子

$$C = \{\mathbf{x} \mid h_1(\mathbf{x}) = 0, \ldots, h_m(\mathbf{x}) = 0, g_1(\mathbf{x}) \leq 0, \ldots, g_r(\mathbf{x}) \leq 0\} \tag{13}$$

- 如果 $C = \mathbb{R}^n$ 那就是不受限

2.1.1 局部最小vs全局最小

- 全局最小 $\mathbf{x}^* : f(\mathbf{x}^*) \leq f(\mathbf{x}) \quad \forall \mathbf{x} \in C$
- 局部最小 $\mathbf{x}^*$： 存在 ε，使得 $f(\mathbf{x}^*) \leq f(\mathbf{x}) \quad \forall \mathbf{x} : \|\mathbf{x} - \mathbf{x}^*\| \leq \varepsilon$
- 使用迭代优化算法来求解，一般只能保证找到局部最小

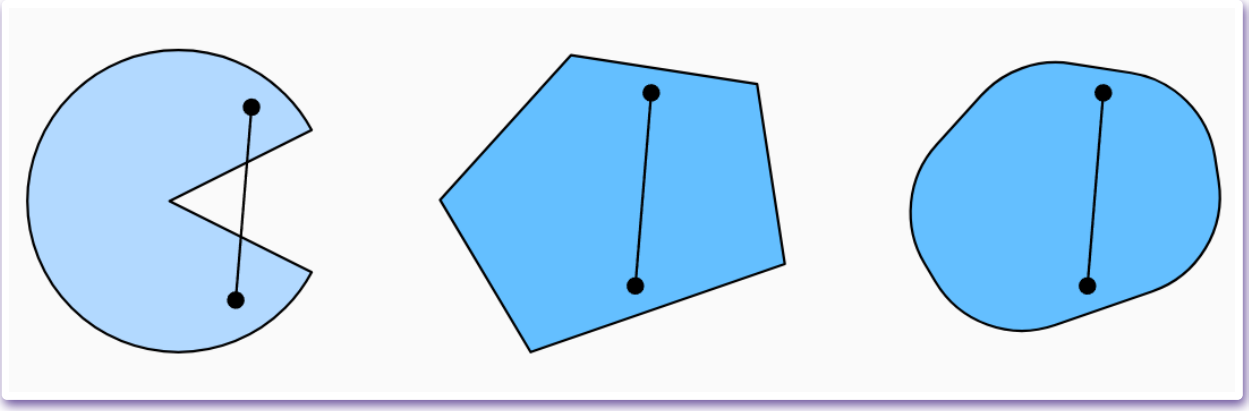


2.1.2 凸集

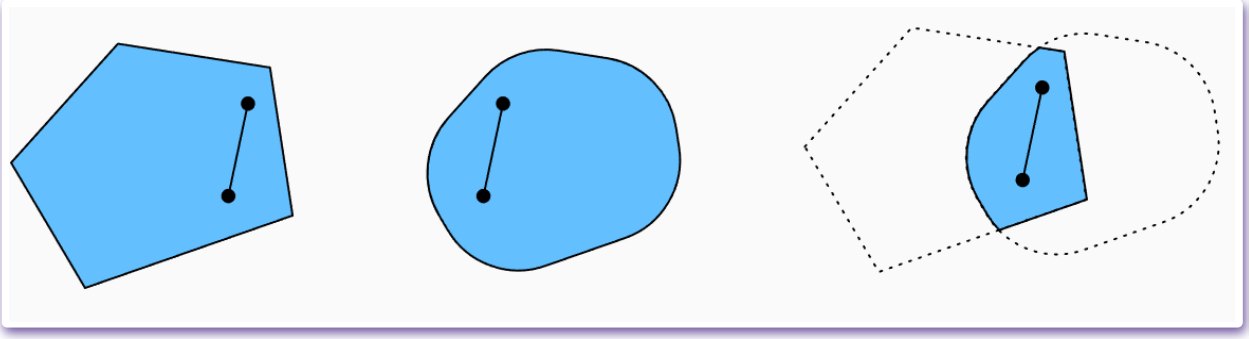
- 一个 $\mathbb{R}^n$ 的子集 C 是凸当且仅当

$$\begin{gathered}\alpha\mathbf{x}+(1-\alpha)\mathbf{y}\in C\\ \forall\alpha\in[0,1]\forall\mathbf{x},\mathbf{y}\in C\end{gathered}\tag{14}$$

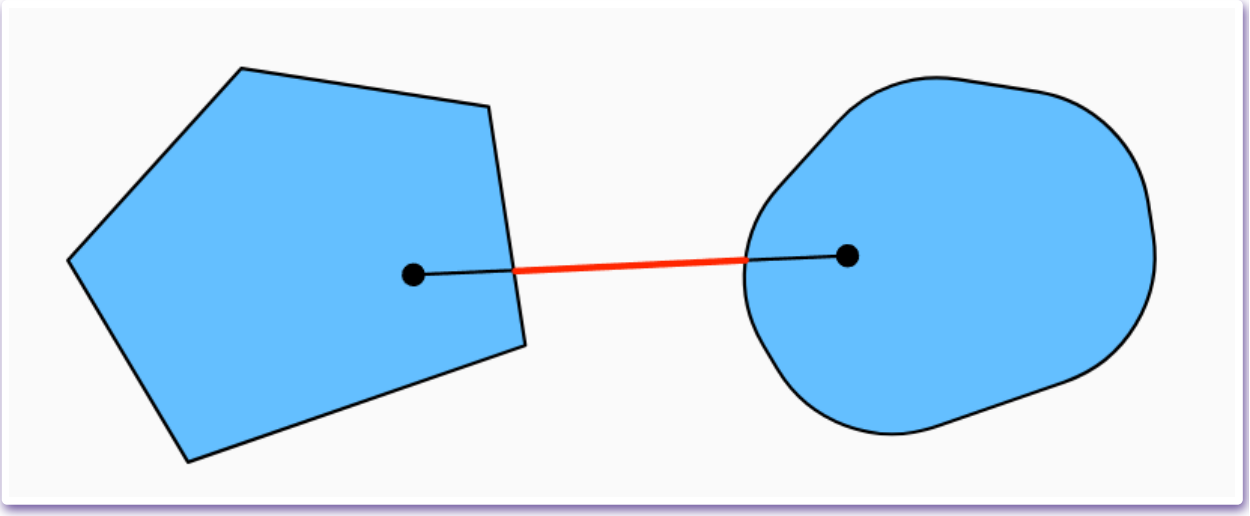
- 第一个是非凸的，后面两个是凸的



- 两个凸集的交集是凸的



- 两个凸集的并集不一定是凸的

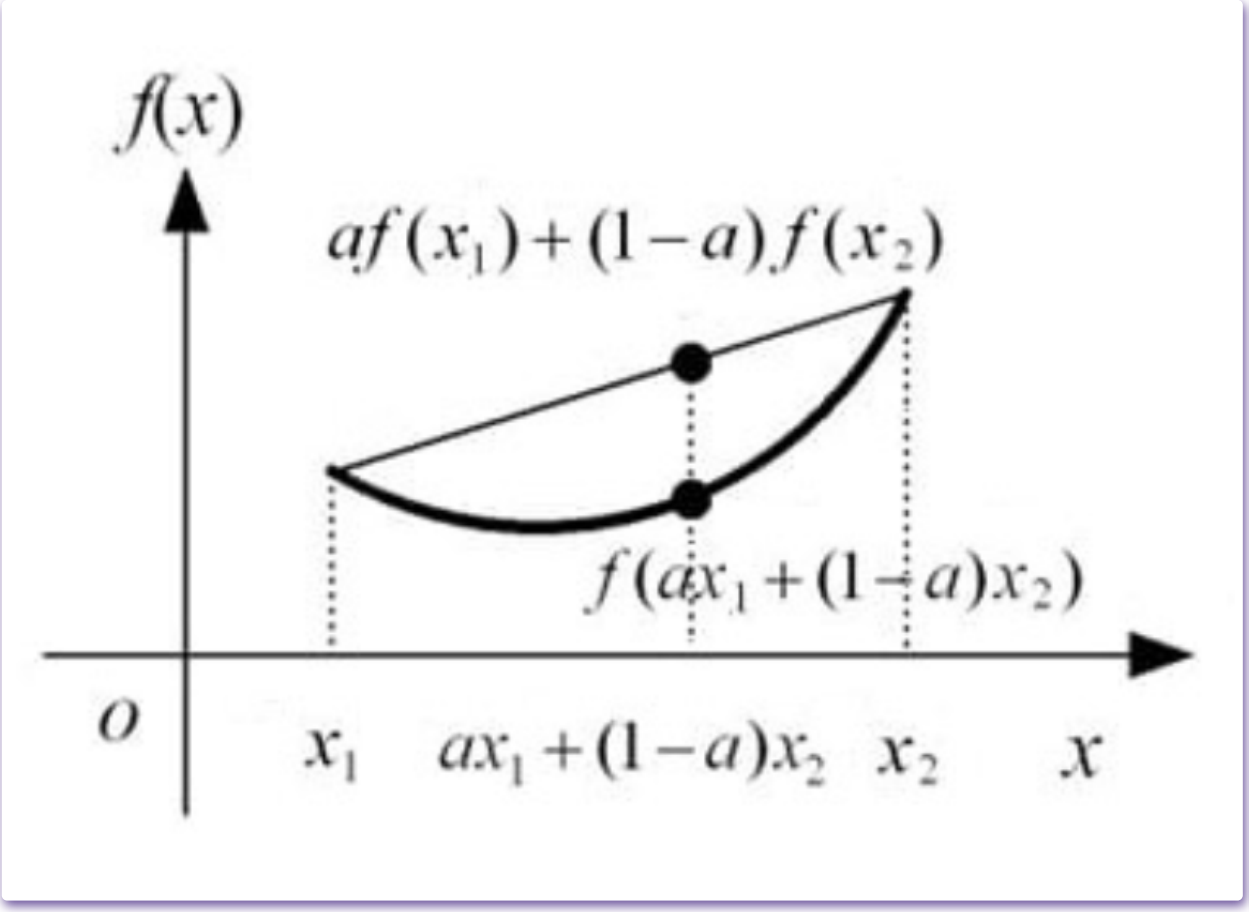


2.1.3 凸函数

- 函数 $f: C \to \mathbb{R}$ 是凸当且仅当

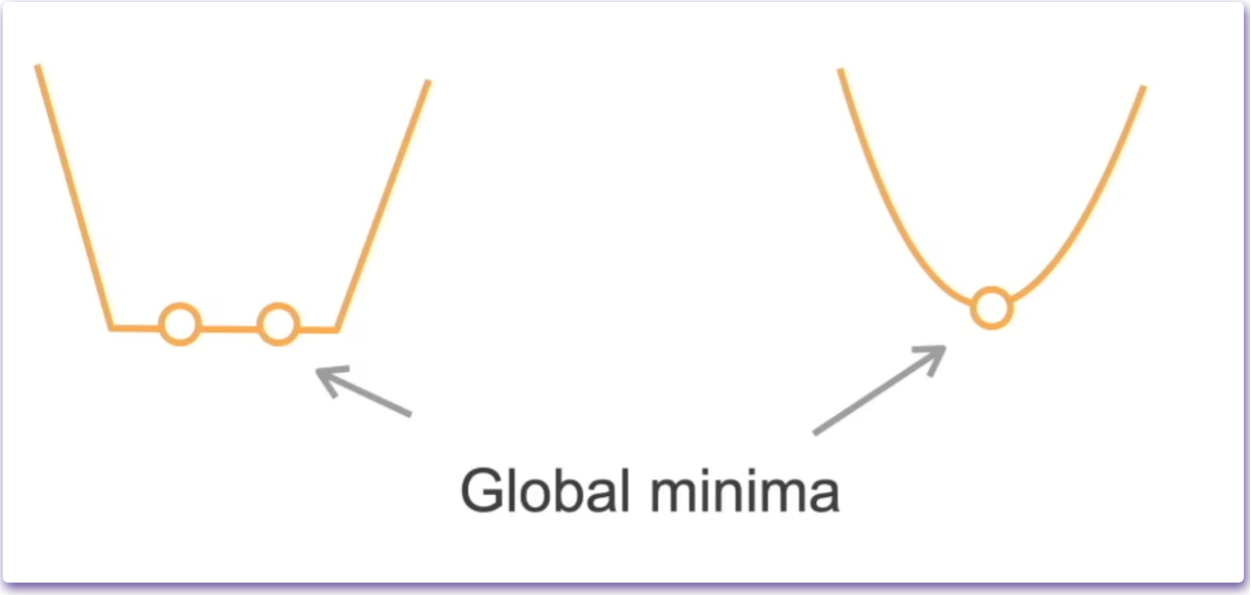
$$\begin{aligned} & f(\alpha \mathbf{x} + (1_0 - \alpha)\mathbf{y}) \quad \leq \alpha f(\mathbf{x}) + (1-\alpha) f(\mathbf{y}) \\ & \forall \alpha \in [0,1] \quad \forall \mathbf{x}, \mathbf{y} \in C \end{aligned} \tag{15}$$

- 如果 $\mathbf{x} \neq \mathbf{y}, \alpha \in (0,1)$ 时不等式严格成立那么叫严格凸函数



2.1.4 凸函数优化

- 如果代价函数 f 是凸的，且限制集合 C 是凸的，那么就是凸优化问题，那么局部最小一定是全局最小
- 严格凸优化问题有唯一的全局最小



- 凸函数和非凸函数的例子

  凸函数

  - 线性回归$f(x) = ||Wx - b||_2^2$
  - Softmax回归

  非凸函数：其他

  - MLP、CNN、RNN、attention

2.2 梯度下降

2.2.1 梯度下降

- 最简单的迭代求解算法
- 选取开始点 $\mathbf{x}_0$，对 $t = 1, \ldots, T$

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \eta \nabla f\left(\mathbf{x}_t\right) \tag{16}$$

  η 叫做学习率

2.2.2 随机梯度下降

- 有 n 个样本时，计算 $f(\mathbf{x}) = \frac{1}{n} \sum_{i=0}^{n} \ell_i(\mathbf{x})$ 的导数太贵，一般不用梯度下降
- 随机梯度下降在时间 t 随机选项样本 t_i 来近似 $f(x)$

$$\begin{aligned} \mathbf{x}_{t+1} &= \mathbf{x}_t - \eta_{t+1} \nabla \ell_{t_i}\left(\mathbf{x}_t\right) \\ \mathbb{E}\left[\nabla \ell_{t_i}(\mathbf{x})\right] &= \mathbb{E}[\nabla f(\mathbf{x})] \end{aligned} \tag{17}$$

2.2.3 小批量随机梯度下降

- 这个基本上是深度学习中真正用的
- 计算单样本的梯度难完全利用硬件资源

- 小批量随机梯度下降在时间 t 采样一个随机子集 $I_t \subset \{1, \ldots, n\}$ 使得 $|I_t| = b$

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \frac{\eta_{t+1}}{b} \sum_{i \in I_t} \nabla \ell_i \left(\mathbf{x}_t\right) \tag{18}$$

  b即是batch_size

- 同样，这是一个无偏的近似，但降低了方差

$$\mathbb{E}\left[\frac{1}{b} \sum_{i \in I_t} \nabla \ell_i(\mathbf{x})\right] = \nabla f(\mathbf{x}) \tag{19}$$

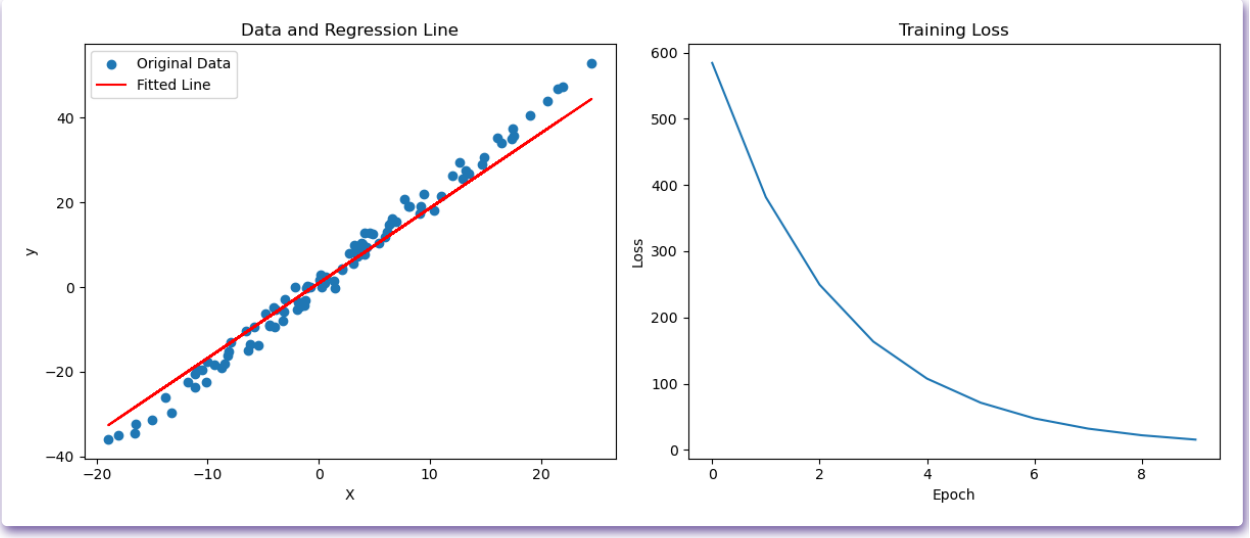
- 例子

```python
import torch
import torch.nn as nn
import matplotlib.pyplot as plt

# 1. 准备数据 ，创建一些样本数据 X 和对应的真实标签 y, y = 2*X + 1 + noise, noise是噪音
X = torch.randn(100, 1) * 10  # 100个样本，每个样本1个特征
y_true = 2*X + 1 + torch.randn(100, 1) * 2 # 加上一些噪声

# 2. 定义模型，使用线性回归模型
class LinearRegressionModel(nn.Module):
    def __init__(self):
        super(LinearRegressionModel, self).__init__()
        self.linear = nn.Linear(1, 1) # 输入维度为1，输出维度为1

    def forward(self, x):
        return self.linear(x)

model = LinearRegressionModel()

# 打印模型初始化的随机参数
print(f"训练前，随机初始化的权重和偏置: {list(model.parameters())}\n")

# 3. 定义损失函数和优化器
learning_rate = 0.001
loss_fn = nn.MSELoss() # 使用均方误差损失
# 使用随机梯度下降 (Stochastic Gradient Descent)
optimizer = torch.optim.SGD(
                            model.parameters(), # 自动获取模型中所有需要训练的参数
                            lr=learning_rate, # 学习率
                           )


# 4. 训练循环
num_epochs = 10  # 训练迭代次数
# 新增一个列表，用于记录每一次的loss值
all_losses = []

for epoch in range(num_epochs):
```

```python
    # forward
    y_pred = model(X)
    # 计算loss
    loss = loss_fn(y_pred, y_true)
    # 记录loss
    all_losses.append(loss.item())

    # -------- 梯度下降三步曲 --------
    # 4.1. 清零过往梯度
    optimizer.zero_grad()
    # 4.2. 计算当前梯度
    loss.backward()
    # 4.3. 更新模型参数
    optimizer.step()

    # 每50步打印损失信息
    if (epoch == 0) or ((epoch + 1) % 2) == 0:
        print(f'Epoch [{epoch+1}/{num_epochs}], Loss: {loss.item():.4f}')


# 5. 查看训练结果
print(f"\n训练后，学习到的权重和偏置，目标权重应接近 [2.]，偏置应接近 [1.]:\n{list(model.parameters())}")

# 使用训练好的模型进行预测
test_input = torch.tensor([[10.0]])
predicted_value = model(test_input).item()
print(f"\n使用训练好的模型预测 x=10 的值，y = {predicted_value:.4f}")

# 6. 可视化结果 (新增部分)
# 关闭梯度计算，以便进行绘图
with torch.no_grad():
    # 创建一个 1 行 2 列的画布，设置画布大小
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))

    # --- 左图：数据散点图和拟合直线 ---
    ax1.set_title('Data and Regression Line')
    ax1.set_xlabel('X')
    ax1.set_ylabel('y')
    # 绘制原始数据 (X, y_true) 的散点图
    # .detach() 是为了从计算图中分离张量，.numpy() 是为了转换为numpy数组以供matplotlib使用
    ax1.scatter(X.detach().numpy(), y_true.detach().numpy(), label='Original Data')
    # 绘制模型学习到的拟合直线
    ax1.plot(X.detach().numpy(), model(X).detach().numpy(), color='red', label='Fitted Line')
    ax1.legend()

    # --- 右图：Loss下降曲线 ---
    ax2.set_title('Training Loss')
    ax2.set_xlabel('Epoch')
    ax2.set_ylabel('Loss')
```

```python
    # 绘制loss随epoch变化的曲线
    ax2.plot(range(num_epochs), all_losses)

    # 调整子图布局，防止标题重叠
    plt.tight_layout()
    # 显示图形
    plt.show()
```

```
"""输出如下"""
训练前，随机初始化的权重和偏置：[Parameter containing:
tensor([[-0.3955]], requires_grad=True), Parameter containing:
tensor([0.8991], requires_grad=True)]

Epoch [1/10], Loss: 584.4702
Epoch [2/10], Loss: 381.5853
Epoch [4/10], Loss: 163.5495
Epoch [6/10], Loss: 71.1610
Epoch [8/10], Loss: 32.0131
Epoch [10/10], Loss: 15.4249

训练后，学习到的权重和偏置，目标权重应接近 [2.]，偏置应接近 [1.]：
 [Parameter containing:
tensor([[1.7706]], requires_grad=True), Parameter containing:
tensor([0.9309], requires_grad=True)]

使用训练好的模型预测 x=10 的值，y = 18.6368
```



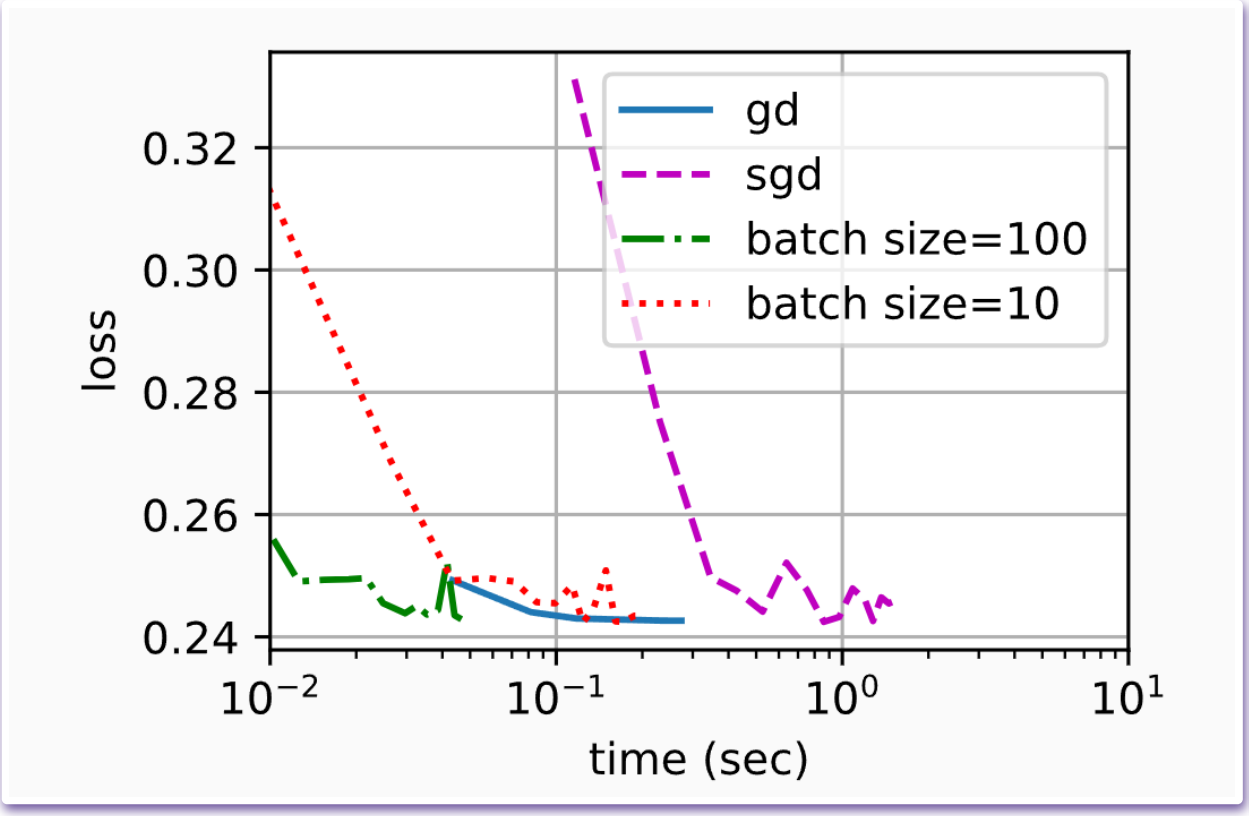
2.2.4 对比

- 下面是各梯度下降的对比

特性	梯度下降 (GD)	随机梯度下降 (SGD)	小批量随机梯度下降 (MB-SGD)
每次更新使用样本数	全部 (m)	单个 (1)	小批量大小 (b), 1<b<m
梯度准确性	精确 (真实梯度)	噪声大 (高方差)	噪声较小 (方差降低)
更新频率 (每epoch)	1	m	m/b
单次更新计算成本	高 (O(m·d))	低 (O(d))	中等 (O(b·d))
每epoch总计算成本	高 (O(m·d))	中等 (m·O(d)=O(m·d)，但串行且低效)	中等 ((m/b)·O(b·d)=O(m·d)，但可并行化)
单次更新内存使用	可能较高 (需访问整个数据集)	低 (单个样本)	中等 (小批量样本)
硬件并行化潜力	有限 (受限于单次大规模计算)	差 (本质上是串行的)	好 (可利用向量化/并行计算)
收敛路径	平滑	震荡	比SGD平滑
跳出局部极小点能力	容易陷入	因噪声可能跳出	可能跳出，但概率低于纯SGD

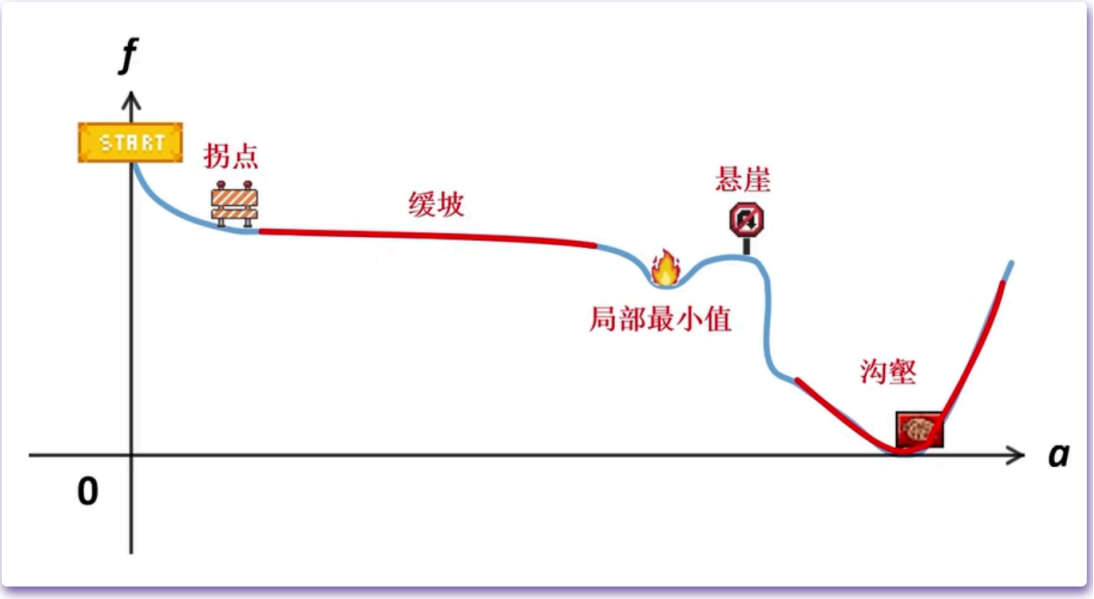
- 以下的图来自李沐的《动手学深度学习》

  可以看出，尽管在处理的样本数方面，随机梯度下降的收敛速度快于梯度下降，但与梯度下降相比，它需要更多的时间来达到同样的损失，因为逐个样本来计算梯度并不那么有效，也没有办法利用硬件资源。 小批量随机梯度下降能够平衡收敛速度和计算效率。 大小为10的小批量比随机梯度下降更有效； 大小为100的小批量在运行时间上甚至优于梯度下降。



2.2.5 原始梯度下降的缺陷

- 原始梯度下降有以下的缺陷
    1. 容易陷入拐点，鞍点，停止更新
    2. **缺乏加速机制**，假设“缓坡”斜率为0.001，且我们给一个较大的学习率0.1,那么每一步走0.0001，假设缓坡长度为10个单位，则走完该“缓坡”需要10万步
    3. **难以跳出局部极小值**，即使是一个很小的沟壑，也不一定能跳出
    4. **无法适应复杂的地形**，以下图为例，可能会在悬崖地形中反复横跳，即在目标点和终点之间反复震荡，难以收敛
    5. **容易产生严重震荡**
    6. 无法自适应调整学习率



- 优化

  原始梯度下降的公式

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \eta \nabla f(\mathbf{x}_t) \tag{20}$$

  我们可以看到每一步梯度更新，与 学习率 η和 当前梯度 $\nabla f(\mathbf{x}_{t-1})$ 相关,所以我们可以分别对这两部分进行优化，即以下两种策略：

    1. 对梯度项进行优化
    2. 对学习率进行优化

2.3 动量法（momentum）

- 动量法是 对梯度进行 优化,类比物理中动量的概念，在梯度更新时，不仅考虑当前位置的梯度，还需要一定程度参考过去的趋势

$$\text{总梯度} = \text{“惯性”} + \text{当前梯度} \tag{21}$$

- 动量法使用平滑过的梯度对权重更新

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \eta m_t \tag{22}$$

  其中 m_t 为动量法平滑过的梯度

    1. pytorch标准动量法

$$\begin{aligned} m_t &= \beta m_{t-1} + \nabla f(\mathbf{x}_t) \\ \mathbf{x}_{t+1} &= \mathbf{x}_t - \eta(\beta m_{t-1} + \nabla f(\mathbf{x}_t)) \end{aligned} \tag{23}$$

    2. 指数加权平均动量法

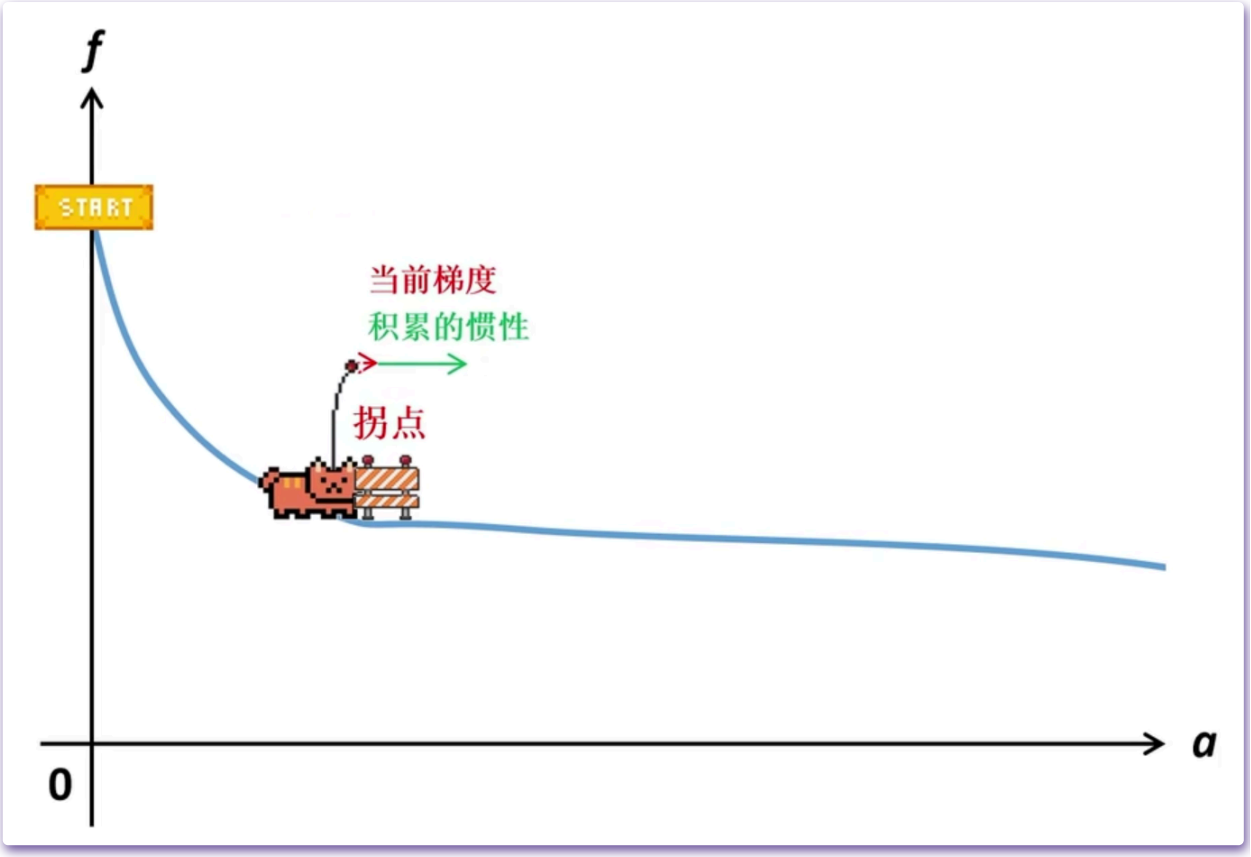
$$\begin{aligned} m_t &= \beta m_{t-1} + (1-\beta) \nabla f(\mathbf{x}_t) \\ \mathbf{x}_{t+1} &= \mathbf{x}_t - \eta((1-\beta) \nabla f(\mathbf{x}_t)) \end{aligned} \tag{24}$$

  β 常见取值［0．5，0．9，0．95，0．99］

  其中指数加权平均动量法的目的是对梯度方向、大小进行平均，平滑梯度，避免震荡

2.3.1 拐点

- 拐点： 当目标点从起点下降到拐点时，如果是原始梯度下降，梯度为0，则梯度不更新了，但是使用动量法，由于在先前下降的过程中累计了部分正向的梯度，因此虽然当前位置的梯度为0，但是经过修正后的梯度并不为0，可以帮助目标点轻松通过拐点



2.3.2 缓坡

- 缓坡：进入缓坡区域的时候，在原始梯度下降，假设“缓坡”斜率为0.001，且我们给一个较大的学习率0.1,那么每一步走0.0001，假设缓坡长度为10个单位，则走完该“缓坡”需要10万步；

$$\mathbf{x}_{t+1} = \mathbf{x}_t - 0.1 * 0.001 \tag{25}$$

使用动量法，让我们在初始位置将积累 的梯度清零，设置$\beta = 0.9$，学习率还是为0.1，缓坡斜率依然为0.001，长度为10个单位

标准动量法，$m_t = 0.9m_{t-1} + 0.0001$

$$\mathbf{x}_{t+1} = \mathbf{x}_t - 0.1 * m_t \tag{26}$$

给定递推关系：

$$m_t = 0.9m_{t-1} + 0.001 \tag{27}$$

1. **齐次解**：
   特征方程 $r = 0.9$，齐次解为 $m_t^{(h)} = C \cdot 0.9^t$。
2. **特解**：
   设常数特解 $m_t^{(p)} = A$，代入得：

$$A = 0.9A + 0.001 \implies A = 0.01 \tag{28}$$

3. **通解**：
   $m_t = C \cdot 0.9^t + 0.01$。
   由初始条件 $m_0 = 0$ 得 $C = -0.01$，故：

$$m_t = 0.01(1 - 0.9^t) \tag{29}$$

4. **化简为目标形式**：
   $0.01 = 10^{-2} = 10^{-3} \cdot 10$，因此：

$$m_t = 10^{-3} \cdot 10(1 - 0.9^t) \tag{30}$$

最终结果：

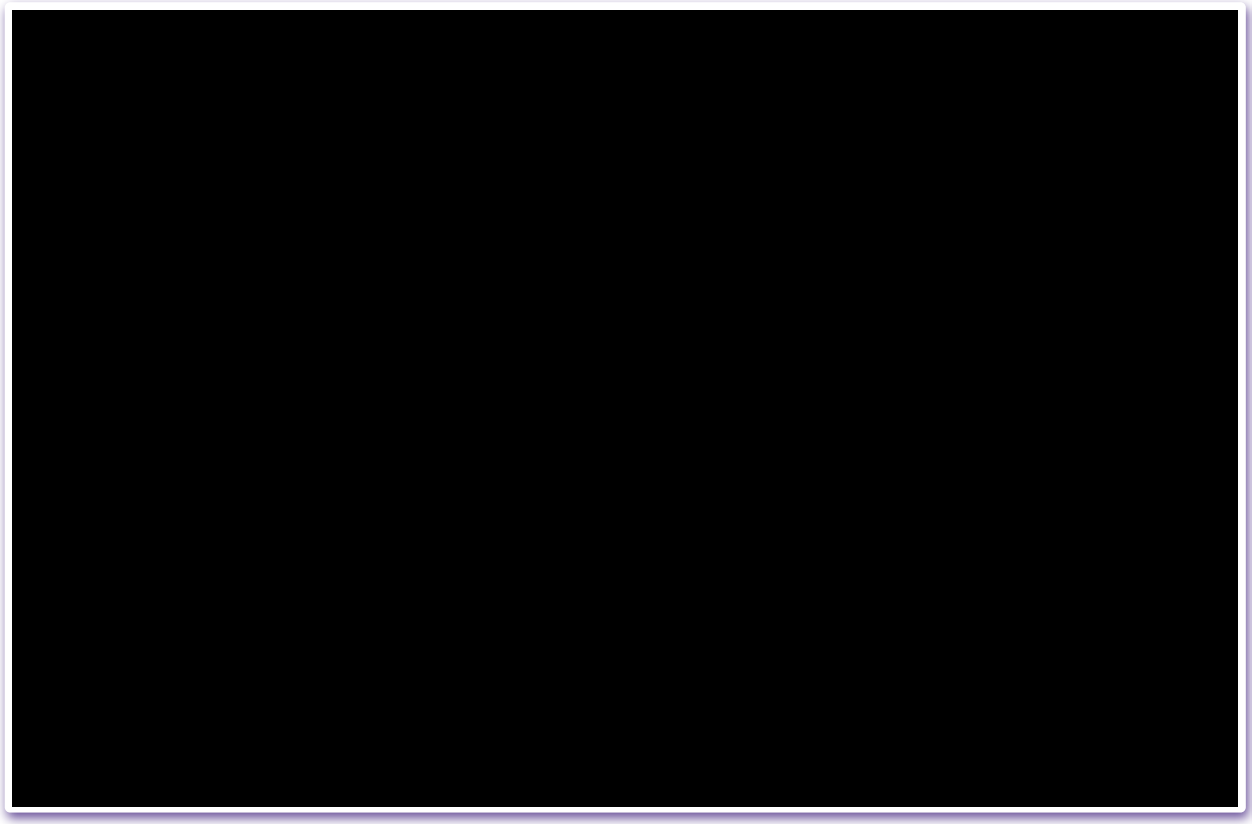
$$m_t = 10^{-3} \cdot 10(1 - 0.9^t) \tag{31}$$

随着速度的增加，目标点最多会被加速到原始速度的10倍，在第t步时走过路程的表达式如下

$$S_t = 10^{-3} * (t + 1) - 0.01(1 - 0.9^{t+1}) \tag{32}$$

即

$$\begin{aligned} m_t &= 10^{-3} \cdot 10(1 - 0.9^t) \\ S_t &= 10^{-3} * (t + 1) - 0.01(1 - 0.9^{t+1}) \end{aligned} \tag{33}$$

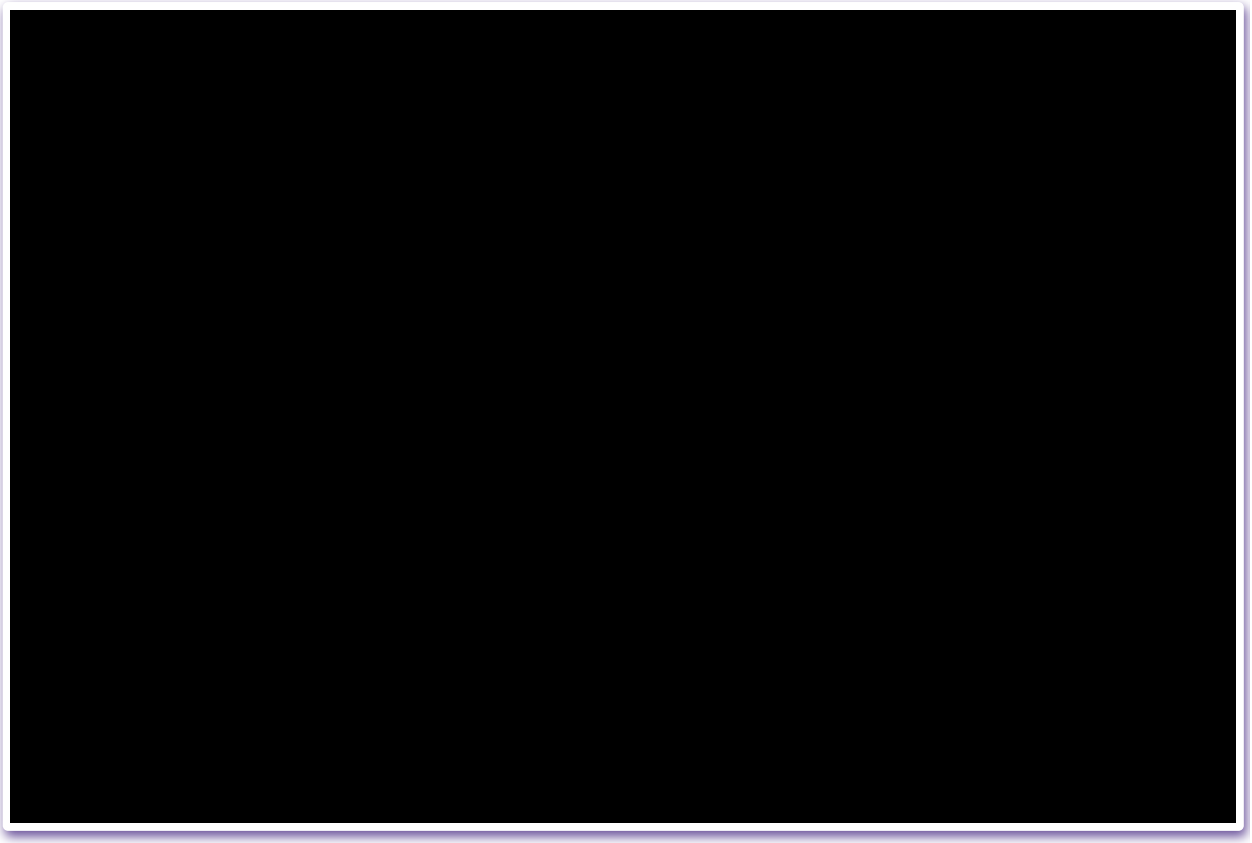


最终相较于原始梯度下降需要的10w步，大约t = 1w时，$S_t = 9.991$,就走完了缓坡

指数加权平均动量法

$$\begin{aligned} m_t &= 0.9m_{t-1} + 0.1 * 0.001 \\ m_t &= 10^{-4} * 10 * (1 - 0.9^t) \\ S_t &= 10^{-4}(t + 1) - 10^{-3}(1 - 0.9^{t+1}) \end{aligned} \tag{34}$$

随着步数的增加，梯度值会趋近于斜率值，不会进一步变大，走完10个单位扔需要10w步

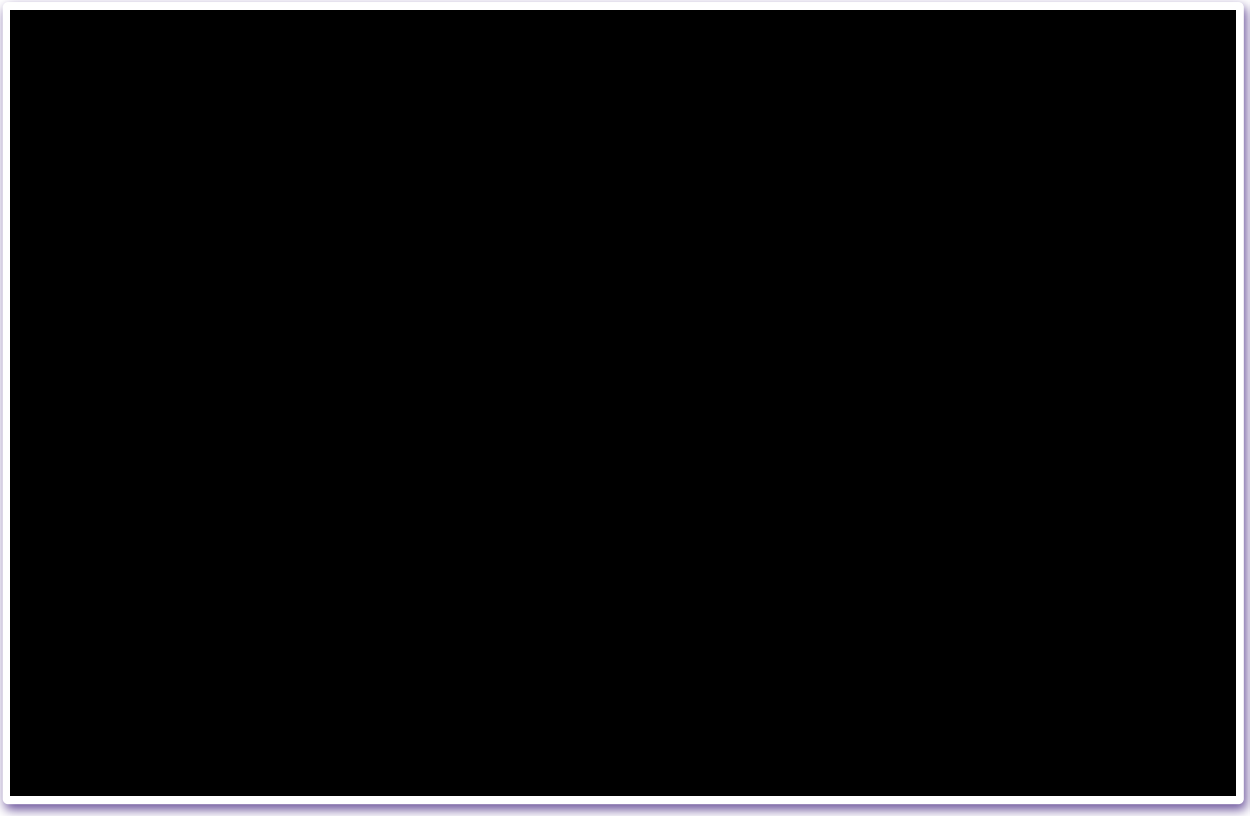


指数加权平均动量法的目的是对梯度方向、大小进行平均，平滑梯度，避免震荡，因此对单方向的长缓坡场景并没有明显的加速效果

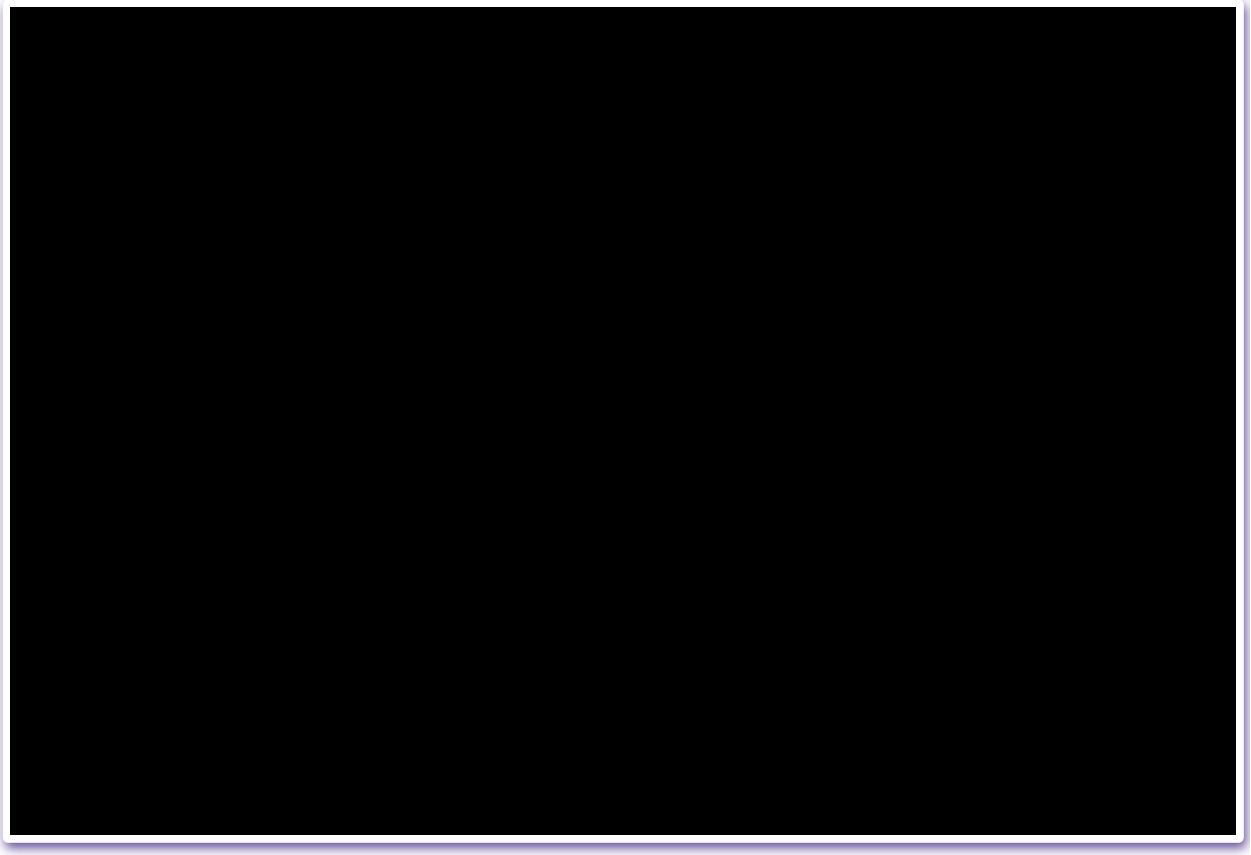
2.3.3 局部极小值

- 局部极小值

  在经过下坡时，因为经过之前的缓坡已经积累了部分梯度值，并且在下坡区间仍然会积累正方向的梯度，所以当到达局部极小值的时候，虽然梯度为0但仍然会向前继续运动，之后虽然会有负方向的梯度抵消之前的功力



但只要先前积累的正向梯度没有消耗殆尽，就可以冲出上坡区域，从而逃离局部极小值位置



2.3.4 悬崖

- 复杂地形：悬崖

  标准动量法:由于每一步都完整贴加了当前梯度，因此较难缓解跳崖问题，甚至由于叠加了过去累计的正向梯度，导致出现更严重的跳崖现象

指数加权平均的动量法，由于当前梯度只有一部分被考虑进来，因此在悬崖边整体梯度并不会迅速变大，但如果后续几部的梯度也较大，整体的梯度也会快速变大，导致任然会发生跳崖现象，只不过产生了一定的滞后

2.3.5 最值点

- 最小值点附近

  当目标点从左侧调到右侧时，由于保留了正向的梯度，叠加上目标点此时的负梯度，整体会削弱向左跳跃的动力，避免了在终点两边反复横跳的现象

2.4 AdaGrad

- AdaGrad是 对学习率进行优化 ,在标准梯度下降法中，学习率是一个定值，会导致无法适应不同特征的梯度变化，在稀疏数据或方向差异大的梯度中，收敛慢或出现震荡。为了解决这个问题，演化出了Adagrad方法。
- AdaGrad公式

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \frac{\eta}{\sqrt{v_t} + \epsilon} \nabla f(\mathbf{x}_t) \tag{35}$$

  其中

  1. v_t 是历史梯度的累计平方和 (注意：是平方和)

$$v_t = \sum_{\tau}^{t} \nabla^2 f(\mathbf{x}_\tau) \tag{36}$$

  2. ϵ 是为防止分母为0的微小值，一般 $\epsilon = 10^{-8}$

- Adagrad对每个参数维护一个历史梯度平方的累计和，使每个参数都有独立的学习率，且会根据历史记录大小自动调整。

2.4.1 拐点

- 拐点

  在从起点逐渐下降到拐点的过程中，学习率逐渐减小，当来到梯度为零的位置时，由于梯度项为零，仍会突然停止更新，提前终止梯度下降过程。

2.4.2 缓坡

- 缓坡

  清空平方和，令缓坡斜率为0.001，η为 0.1。可以看到在进入缓坡时

$$\begin{aligned} v_t &= \sum_{\tau}^{t} \nabla^2 f(\mathbf{x}_\tau) \\ v_1 &= 0.001^2 \\ &= 10^{-3} \end{aligned} \tag{37}$$

带入公式，学习率这一项被扩大了1000倍；后续随着步数增加，学习率逐渐缩小到1000除以根号二倍，1000除以根号三倍。对于一段十个单位的距离，最终大概只需要不到3000步就可以走完。

对于短时间的训练，该方法确实可以加速收敛，但是由于是对历史梯度平方的简单累加，分母部分会难以避免的越变越大，导致学习率单调变小，最终学习率几乎为零，模型失去学习能力

例如当缓坡长度增加到1万个单位时，经过计算，当目标点移动到第十的六次方步时，分母项变为一，此时目标点仅运动了2000个单位左右，之后的移动均不会再有加速效果，而是不断的减速，

最终完整走完1万个单位，需要大概$2.5*10^9$方步，远大于原始梯度下降的十的八次方步。

2.4.3 局部极小值

- 局部极小值

  当进入局部最小值区域后，由于梯度趋近于零，学习率也因累计梯度而越来越小，因此训练后期进入局部最小值区域时，模型容易陷进去。

2.4.4 悬崖

- 复杂地形：悬崖

  来到悬崖边，由于在前几步梯度值会非常大，会导致学习率被快速缩小，导致整体步长减小，从而一定程度上缓解跳崖问题。

2.4.5 最值点

- 最小值点附近

  来到终点附近，当目标点在终点两边震荡时，虽然不像动量法那样可以有相反方向的梯度对当前梯度进行抵消，但是由于在沟壑两侧梯度较大，学习率随着分母累加会逐渐变小，因此仍能一定程度的减小震荡，加速收敛。

2.4.6 缺陷

- Adagrad对学习率的修正过程同样存在着明显的缺陷。由于Adagrad中累计梯度平方的gt会单调递增，导致学习率持续下降，最终趋近于零，模型会过早停止学习。因此当梯度下降陷入局部最小值区域，训练非常容易陷入停滞。
- 为了解决上述问题，人们在Adagrad的基础上又提出了RMSprop算法。

2.5 RMSProp

- 为解决AdaGrad陷入局部极值点，在Adagrad的基础上又提出了RMSprop算法，其 核心思想 ：使用指数加权移动平均替换掉Adagrad中对梯度的平方的简单累加

$$\mathbf{x}_{t+1}=\mathbf{x}_t-\frac{\eta}{\sqrt{v_t}+\epsilon}\nabla f\left(\mathbf{x}_t\right) \tag{38}$$

  其中

  1. v_t 替换AdaGrad中的梯度累计平方和为数加权移动平均

$$v_t=\beta v_{t-1}+(1-\beta)\nabla^2 f\left(\mathbf{x}_t\right) \tag{39}$$

2. ϵ 是为防止分母为0的微小值，一般 $\epsilon = 10^{-8}$

- RMSprop更关心最近的行为，如果最近梯度波动大就慢一点，如果最近很平稳就快一点

2.5.1 缓坡

- 缓坡

  使用RMSprop算法再来看一次在缓坡区域的梯度下降过程, 缓坡斜率 $\nabla f(\mathbf{x}_t)$= 0.001，$\beta = 0.9$, 学习率$\eta = 0.1$

$$\begin{aligned} \mathbf{x}_{t+1} &= \mathbf{x}_t - \frac{\eta}{\sqrt{v_t} + \epsilon} \nabla f(\mathbf{x}_t) \\ &= \mathbf{x}_t - \frac{0.1}{\sqrt{v_t} + \epsilon} * 0.001 \end{aligned} \tag{40}$$

  此时

$$\begin{aligned} v_t &= \beta v_{t-1} + (1-\beta) \nabla^2 f(\mathbf{x}_t) \\ &= 0.9 v_{t-1} + (1-0.9) * 0.001^2 \\ &= 10^{-7} * 10 * (1-0.9^t) \end{aligned} \tag{41}$$

  随着步数增加，虽然开始时的步长也会减小，但是最终分母值会逐渐逼近于当前梯度值，不会出现Adagrad中学习率越变越小直到消失的情况。即当 t 越来越大时，0.9^t 趋于0，此时

$$\begin{aligned} v_t &= 10^{-7} * 10 * (1-0.9^t) \\ &= 10^{-6} \end{aligned} \tag{42}$$

  动态学习率

$$\begin{aligned} \frac{0.1}{\sqrt{v_t} + \epsilon} &= \frac{1}{\sqrt{10^{-6}} + \epsilon} \\ &\approx \frac{0.1}{0.001} \\ &\approx 100 \end{aligned} \tag{43}$$

  最终会稳定在100附近

2.5.2 对比RMSProp和指数加权平均动量法

- RMSProp和momentum对比

	momentum	RMSprop
鞍点	可以有效地逃离鞍点	难以逃离鞍点
缓坡	无法在缓坡上进行加速	通过控制学习率实现在缓坡上的加速
局部极小	具有一定跳出局部最小值的能力	跳出局部最小值能力较弱
悬崖	无法有效缓解跳崖现象	能够有效缓解跳崖现象
沟壑	可以缓解震荡，加速收敛	可以缓解震荡
动态调整学习率	否	是
多维度动态调整学习率	否	是

- 既然两种方法具有互补的特性，那么我们为什么不将两种方法结合起来，同时调节梯度项和学习率项呢？让动量法仅对一段时间内的梯度进行加权平均，将调节学习率与影响加速减速的任务交给RMSprop。基于这种想法衍生出来的方法就是Adam。

2.6 Adam

- 调节梯度项和学习率项，基于这种想法衍生出来的方法是Adam。
- Adam表达式

$$
\begin{aligned}
&\mathbf{x}_{t+1}=\mathbf{x}_t-\frac{\eta}{\sqrt{v_t}+\epsilon} m_t \\
&v_t=\beta_1 V_{t-1}+\left(1-\beta_1\right) \nabla^2 f\left(\mathbf{x}_t\right) \\
&m_t=\beta_2 m_{t-1}+\left(1-\beta_2\right) \nabla f\left(\mathbf{x}_t\right)
\end{aligned}
\tag{44}
$$

注意，m_t 中的梯度是没有平方的

偏差修正

$$
\begin{aligned}
&\mathbf{x}_{t+1}=\mathbf{x}_t-\frac{\eta}{\sqrt{\hat{v}_t}+\epsilon} \hat{m}_t \\
&v_t=\beta_1 V_{t-1}+\left(1-\beta_1\right) \nabla^2 f\left(\mathbf{x}_t\right) \\
&m_t=\beta_2 m_{t-1}+\left(1-\beta_2\right) \nabla f\left(\mathbf{x}_t\right) \\
&\hat{v}_t=\frac{v_t}{1-\beta_1^t} \\
&\hat{m}_t=\frac{m_t}{1-\beta_2^t}
\end{aligned}
\tag{45}
$$

- 观察Adam的表达式，可以看到由于在v_t和m_t中都用到了指数加权平均，这种计算方法会导致在训练的前几步，无论是v_t还是m_t的值都会偏小。

  例如在缓坡中，缓坡梯度0.001，beta为0.9，梯度的期望应该与斜率一致，mt计算出来的第一

步的值仅为斜率的1/10，可能会使训练初期不稳定，造成震荡或收敛困难等问题。

因此通常需要额外对VT和mt进行偏差修正，解决前期估计值偏小的问题，使得即使在前几步vt和mt较小，也能拉回真实的统计量期望。

2.6.1 拐点

- 当来到鞍点时，梯度修正项mt会在惯性助力下顺利通过鞍点。

2.6.2 缓坡

- 当来到缓坡区域时，学习率修正项v_t会使目标点加速通过。

2.6.3 局部极小值

- 当来到局部最小值区域时，m_t项中累积的惯性以及v_t项在平缓区域对学习率的动态增大，会共同协助目标点更大希望的跳出局部最小值区域。

2.6.4 悬崖

- 来到悬崖区域，受到梯度瞬间增大的影响，v_t会迅速减小，导致学习率随之减小，从而一定程度缓解了跳崖问题。

2.6.5 最值点

- 来到终点附近，当目标点开始在终点两边震荡时，受m_t影响，正反方向的梯度会进行一定程度的相互抵消，会迅速减小震荡，加速收敛。

2.6.6 扩展到三维

- 多参数下，扩展到三维，在a方向上m_t可以减小震荡，加速收敛，B方向受到v_t影响，学习率增大，加速下降。同时a、b方向都能自适应的调整学习率大小，使得针对多个参数的训练更加灵活。
- 然而虽然Adam看似具有了很大的优势，但在实际训练中并不一定Adam就是最好的选择。有时候选择标准动量法或RMSprop反而会有更好的效果。Adam是比较常用的，但是不是必须要用的，因为sgd+momentum 效果已经非常好了，Adam不一定有sgd+momentum 好，但是Adam对学习率不敏感，所以实践中还是要多尝试